

Pointers in C — Annotated Snippets

The main syntaxes to remember:

- `&x` — **address-of**: gives the memory address where the variable `x` is stored.
- `int* p` — this is a **declaration**, `*` means "p is a pointer to int". This is the same as `int *p`, which is more commonly used.
- `*p` — in an **expression**, `*` means **dereference**: go to the address stored in `p` and read/write the int there. So `*p` basically is another way to refer to the variable that `p` points to. Note that the addresses shown in outputs are examples. Actual values differ on every run (the OS randomises stack/heap placement).

1. Address-of and dereference

`int *p` (or `int* p` - both are the same) declares a pointer; `*p = 20` writes through it. Since `p` holds the address of `x`, writing to `*p` writes to `x`.

```
#include <stdio.h>
int main() {
    int x = 10;
    int *p = &x;    // & : address-of. p now holds x's address
    *p = 20;        // * : dereference. "go to that address, treat as int"
                   // x is now 20
    printf("%d", x);
}
```

Output:

```
20
```

`x` was never assigned directly after `10`, yet it prints `20`. Two names (`x` and `*p`) refer to the same memory location.

2. `&a` and `pa` are the same; `a` and `*pa` are the same

If `pa` was set to `&a`, then `pa` and `&a` should print the same address, and `*pa` and `a` should print the same value. Let's verify both claims in one program.

```
#include <stdio.h>
int main() {
    int a = 3;
    int *pa = &a;

    printf("%p %p \n", &a, pa);
    printf("%d %d", a, *pa);
    return 0;
}
```

Output (address varies):

```
0x7ffee4a3b99c 0x7ffee4a3b99c
3 3
```

First line: two identical addresses — `pa` holds exactly what `&a` produces. Notice the form of the address: `0x7ffee4a3b99c`. Memory addresses are conventionally written in **hexadecimal** (base 16: digits `0–9` then `a–f`, with a `0x` prefix).

`%p` is the `printf` specifier for printing an address in this form; we'll look at it, and at what goes wrong with other specifiers, in snippets 14–16.

Second line: two identical values — dereferencing `pa` reads `a`. This is the whole idea of a pointer in two lines: `pa` is the address, `*pa` is the variable.

Optional detailed note on hex: Hex fits memory naturally because each hex digit is exactly 4 bits, so two hex digits make one byte — a 12-hex-digit address is 48 bits. Why 48 and not 64? A pointer has room for 64

bits, but 2^{64} bytes is 16 exabytes — no machine has anything close — so current processors only implement 48-bit virtual addresses ($2^{48} = 256$ TB, plenty). The unused top 16 bits are zero for user programs, and `%p` doesn't print leading zeros, so `0x7ffee4a3b99c` is really `0x00007ffee4a3b99c`. That's also why stack addresses on an Intel Mac always begin with `0x7ff`: user space ends at `0x00007fffffffffff`.

3. Pass by value — `sum`

Before mixing pointers with functions, recall how an ordinary function call works. When we call `sum(a, b)`, does `sum` receive the variables `a` and `b` themselves, or something else? Also notice `scanf` needs `&a` and `&b` — why does it need addresses when `sum` doesn't?

```
#include <stdio.h>
int sum(int a, int b)
{
    return a+b;
}

int main() {
    int a,b;
    scanf("%d %d", &a,&b);
    int c = sum(a, b);
    printf("%d", c);
}
```

Input / Output:

```
4 5
9
```

`sum` receives *copies* of `a` and `b`. That's fine here because `sum` only reads them and sends the answer back via `return`. `scanf` is different: its job is to *write* into `a` and `b`, so a copy would be useless — it needs to know where the real variables live, hence `&a`, `&b`. Keep this contrast in mind for the next few snippets.

4. Pass by value — why `increment` doesn't work

Suppose we want a function that adds 1 to a variable. The obvious attempt is below. Will `a` in `main` change?

```
#include <stdio.h>
void increment(int a)
{
    a = a+1;
    return;
}

int main() {
    int a;
    scanf("%d", &a);
    increment(a);
    printf("%d", a);
}
```

Input / Output:

```
5
5
```

No change. `increment` received a *copy* of `a`, incremented the copy, and the copy was destroyed when the function returned. `main`'s `a` was never touched. The function "ran" but had no visible effect.

5. Passing a pointer — but incrementing the pointer, not the value

The lesson from `scanf` was: to let a function modify a variable, give it the variable's address. So let's change the parameter to `int *a` and call `increment(&a)`. Is this enough?

```
#include <stdio.h>
void increment(int *a)
{
    a = a+1;
    return;
}

int main() {
    int a,b;
    scanf("%d", &a);
    increment(&a);
    printf("%d", a);
}
```

Input / Output:

```
5
5
```

Still no change, but for a different reason. Inside the function, `a` is a pointer, and `a = a+1` modifies *the pointer* — it now points 4 bytes further along (pointer arithmetic on `int*` moves by `sizeof(int)`). We never dereferenced, so the `int` that `a` points to was never read or written. And since the pointer itself was passed by value, moving the local copy of it has no effect on `main` either.

6. Passing a pointer — dereferencing to modify the caller's variable

We have the address; we just need to *go to it*. That's what `*` does in an expression. Let's write `*a = *a + 1`.

```
#include <stdio.h>
void increment(int *a)
{
    *a = *a+1;
    return;
}

int main() {
    int a,b;
    scanf("%d", &a);
    increment(&a);
    printf("%d", a);
}
```

Input / Output:

```
5
6
```

It works. `*a` on the right reads the `int` at the address; `*a` on the left writes it back to the same address. That address is `main`'s `a`. This is the standard C pattern for letting a function modify a caller's variable — sometimes called "pass by reference", though really it's still passing a value, just a value that happens to be an address. It's exactly what `scanf` does.

7. `sum` with pointer parameters

Can we write `sum` this way too, with pointers? Yes. Should we?

```
#include <stdio.h>
int sum(int *a, int *b)
{
    return *a + *b;
}

int main() {
    int a,b;
    scanf("%d %d", &a,&b);
    int c = sum(&a, &b);
    printf("%d", c);
}
```

Input / Output:

```
4 5
9
```

Same answer as snippet 3. `*a + *b` reads the two ints through the pointers and adds them. But since `sum` only reads, passing copies (snippet 3) was already fine — the pointer version is extra work for no gain. Use pointers when a function needs to *modify* the caller's data, or when copying would be expensive. The next snippet is where that second reason shows up.

8. Arrays are passed as pointers

What happens when we pass an *array* to a function? Is the whole array copied like an int? Watch what happens to `num[1]` after the call.

```
#include <stdio.h>
int sum(int A[], int n)
{
    int total = 0;
    for(int i=0 ; i < n; i++)
    {
        total += A[i];
    }
    A[1] = 100;
    return total;
}

int main() {
    int num[5] = {1,2,3,4,5};
    int c = sum(num, 5);
    printf("%d",num[1]);
    return 0;
}
```

Output:

```
100
```

The array was *not* copied. The `A[1] = 100;` inside the `sum` function actually modified the original array itself! When you pass an array name to a function, C passes a **pointer to its first element** — so `int A[]` in the parameter list is really `int *A`. `A[1] = 100` inside the function wrote straight into `main`'s `num`. (`c` receives 15 but is never printed.) This is also why the function needs the extra `n` argument: a pointer carries no length information, and so doesn't know where the array ends - only where it begins.

9. Pointer with [] indexing

If an array name is basically a pointer to its first element, can we store it in a pointer variable and then use `[]` on that pointer? Note there's no `&` in `int *p = num;`.

```
#include <stdio.h>
int main() {
    int num[5] = {1, 2, 3, 4, 5};
    int *p = num;

    for (int i = 0; i < 5; i++)
    {
        printf("%d", p[i]);
    }
    return 0;
}
```

Output:

```
12345
```

Yes. `num` on its own "decays" to `&num[0]` (i.e. the address of the first element of the array), so `p` now points at the first element, and `p[i]` reads element `i` exactly like `num[i]`. The `[]` operator isn't special to arrays — it works on any pointer.

10. `p[i]` is `*(p+i)`

What does `p[i]` actually mean to the compiler? Let's write out the pointer arithmetic by hand and see if we get the same result.

```
#include <stdio.h>
int main() {
    int num[5] = {1, 2, 3, 4, 5};
    int *p = num;

    for (int i = 0; i < 5; i++)
    {
        printf("%d", *(p+i));
    }
    return 0;
}
```

Output:

```
12345
```

Identical. `p + i` is the address `i` ints after `p` (`i * sizeof(int)` bytes further), and `*(p+i)` reads the int there. The C language *defines* `A[i]` as `*(A + i)` — the two forms are interchangeable everywhere, and the next two snippets look at that arithmetic more closely.

11. Pointer arithmetic scales by element size

We keep saying `p+1` is "4 bytes further" for an `int *`. Is the 4 special, or does it depend on what `p` points to? Let's do `+1` on pointers to three different types and print the before/after addresses.

```
#include <stdio.h>
int main() {
    int    i_arr[3];   int    *ip = i_arr;
    char   c_arr[3];   char    *cp = c_arr;
    double d_arr[3];   double *dp = d_arr;

    printf("%p %p\n", ip, ip + 1);
    printf("%p %p\n", cp, cp + 1);
    printf("%p %p\n", dp, dp + 1);
    printf("%zu %zu %zu\n", sizeof(int), sizeof(char), sizeof(double));
    return 0;
}
```

Output (addresses vary):

```
0x7ffee4a3b9a0 0x7ffee4a3b9a4
0x7ffee4a3b99d 0x7ffee4a3b99e
0x7ffee4a3b980 0x7ffee4a3b988
4 1 8
```

+1 moved the `int *` by 4, the `char *` by 1, and the `double *` by 8 — exactly `sizeof` of the pointed-to type in each case.

So pointer arithmetic is always in units of *elements*, never bytes: `p + i` is really the address `p + i * sizeof(*p)`. This is why the type of a pointer matters even though every pointer is just an 8-byte address — the type tells the compiler how far to step (and how many bytes to read when you dereference). (`%zu` is the specifier for `sizeof` results.)

12. Subtracting and comparing pointers

If `p + 3` steps forward 3 elements, does `q - p` give back a number of elements? And can we compare two pointers? Let's try, using pointers into the same array.

```
#include <stdio.h>
int main() {
    int num[5] = {10, 20, 30, 40, 50};
    int *p = &num[1];
    int *q = &num[4];

    printf("%ld\n", (long)(q - p)); // how many elements apart?
    printf("%ld\n", (long)(q - num)); // index of the element q points to
    printf("%d %d\n", *(q - 1), *(p + 2));
    printf("%d\n", q > p);
    return 0;
}
```

Output:

```
3
4
40 40
1
```

`q - p` is 3, not 12. `p` and `q` are 12 bytes apart in memory (three `ints` × 4 bytes), but the compiler divides that byte difference by `sizeof(int)` and reports the answer in *elements*. This makes subtraction the exact inverse of addition: `p + 3` means "3 elements forward", and `(p + 3) - p` gives back 3 — the same number you added. The language keeps addition, subtraction, indexing and `++` all in units of elements, so you never have to think in bytes.

`q - num` gives the index of the element `q` points to — a handy trick when a function hands you a pointer into an array and you want its position.

`*(q - 1)` and `*(p + 2)` both reach `num[3]`.

And `q > p` is 1 (true) because `q` points further along.

All of this is only meaningful when both pointers point into the *same* array (the reverse function in snippet 17 relies on exactly that); subtracting or comparing pointers into unrelated objects is undefined behaviour.

13. Walking a pointer with `p++`

Instead of recomputing `p+i` every iteration, can we just move `p` itself one step each time?

```
#include <stdio.h>
int main() {
    int num[5] = {1, 2, 3, 4, 5};
    int *p = num;

    for (int i = 0; i < 5; i++)
    {
        printf("%d", *p);
        p++;
    }
    return 0;
}
```

Output:

```
12345
```

Same output. `p++` advances by one `int` (4 bytes), not one byte. After the loop, `p` points one past the end of the array — that's legal to *hold* but not to dereference. Note that `num++` would be a compile error: an array name is not a modifiable pointer, which is why we copy it into `p` first. Using `num` in arithmetic *without* modifying it is fine, though — `*(num+i)` and `num[i]` both work, because `num` decays to a pointer to its first element in any expression; the only thing you can't do is assign to `num` itself (`num++`, `num = p`).

14. Printing a pointer with `%d` (wrong)

Let us now try to print the pointer and see what happens. We know that `p` stores the address of the variable it is pointing to. And since the entire memory is like a huge array, each address is just a number (like index in an array). So let's just print the pointer as an `int` and see what happens.

```
#include <stdio.h>
int main() {
    int x = 10;
    int *p = &x;
    *p = 20;
    printf("%d", p);
}
```

Output (garbage, varies):

```
-1345849236
```

Compile warning: format '`%d`' expects argument of type '`int`', but argument has type '`int *`'.

This is clearly garbage value, since address can't be negative. What went wrong?

`p` has an address, not an `int`. `%d` tells `printf` to expect an `int`, so this is a type mismatch — undefined behaviour. The compiler warns about it. On a typical 64-bit Mac/Linux, each address is 8 bytes (ie. 64 bits) long, whereas `int` is typically 4 bytes (ie. 32 bits) long. And so you'll get some meaningless number (often the low 32 bits of the address, reinterpreted as a signed `int`). It changes every run.

15. Printing a pointer with `%p` (correct)

So how do we print an address properly? C has a format specifier made specifically for pointers: `%p`. Let's use that instead.

```
#include <stdio.h>
int main() {
    int x = 10;
    int *p = &x;
    *p = 20;
    printf("%p", p);
}
```

Output (example, varies):

```
0x7ffee4a3b9ac
```

Now we get a sensible address, printed in hexadecimal (which is how memory locations are referred to). This is *where* `x` lives in memory, not its contents. Run the program twice and the address will differ; that's the OS placing the stack at a different location each run, and is nothing to worry about.

16. Can I use `%lld` instead of `%p`?

Addresses are 8 bytes, and so is a `long long`. So can we skip `%p` and just print the pointer with `%lld` to see the address as a normal decimal number? Let's try it both with and without a cast.

```
#include <stdio.h>
int main() {
    int x = 10;
    int *p = &x;
    printf("%p\n", p);
    printf("%lld\n", p);
    printf("%lld\n", p+1);
    printf("%lld\n", (long long)p);    // cast, then %lld
    printf("%lld\n", (long long)(p+1)); // the next int's address
}
```

Output (address varies):

```
0x7ffee4a3b9ac
140732820683180
140732820683184
140732820683180
140732820683184
```

Compile warnings (two of them, for lines 2 and 3): `format specifies type 'long long' but the argument has type 'int *' [-Wformat]`.

All four decimal lines look right — lines 2 and 4 are the same number as line 1 in decimal, and lines 3 and 5 are 4 higher because `p+1` is the next `int` (+4 for a +1, as we saw in snippet 11). So why does the compiler complain about lines 2 and 3 but not 4 and 5?

Because the rule for `printf` is that the argument's *type* must match the format specifier, and `int *` is not `long long` — same mistake as `%d` in snippet 14, so it's undefined behaviour. It only *happens* to print correctly here because on a 64-bit system a pointer and a `long long` are both 8 bytes and are passed to `printf` the same way, so `printf` reads the right bytes. The cast in lines 4 and 5 converts the pointer to an actual `long long` before the call, so the type now matches and the warning goes away. Even then, this relies on `long long` being wide enough to hold an address — true on every machine you'll use in this course, but not guaranteed by C. `%p` remains the default: it's the only specifier that accepts a pointer directly.

17. Two-pointer reverse

Putting pointer arithmetic to work: reverse an array using two pointers that start at opposite ends and walk toward each other. What does `num` hold afterwards?

```
#include <stdio.h>
void reverse(int *A, int n)
{
    int *left = A;
    int *right = A + n - 1;
    while (left < right)
    {
        int temp = *left;
        *left = *right;
        *right = temp;
        left++;
        right--;
    }
}

int main() {
    int num[6] = {1, 2, 3, 4, 5, 6};
    reverse(num, 6);
    return 0;
}
```

Output:

```
(nothing printed)
```

There's no `printf`, but `num` is now `{6, 5, 4, 3, 2, 1}`. `A + n - 1` is the address of the last element. Each iteration swaps `*left` and `*right`, then moves both inward. Comparing pointers (`left < right`) is valid because both point into the same array; the loop stops when they meet or cross. Because `reverse` received a pointer to `main`'s array (snippet 8), the swaps happened in place.

18. `int *A` vs `int A[]` as a parameter — identical

Snippet 8 declared the parameter as `int A[]`. Does declaring it as `int *A` change anything?

```
#include <stdio.h>
int sum(int *A, int n)
{
    int total = 0;
    for(int i=0 ; i < n; i++)
    {
        total += A[i];
    }
    A[1] = 100;
    return total;
}

int main() {
    int num[5] = {1,2,3,4,5};
    int c = sum(num, 5);
    printf("%d",num[1]);
    return 0;
}
```

Output:

```
100
```

Nothing changes. In a parameter list, `int A[]` and `int *A` mean exactly the same thing: a pointer to `int`. Indexing with `A[i]` on a pointer parameter is perfectly normal (snippet 9).

19. Pointer parameter, pointer-arithmetic access

And if we also swap the access from `A[i]` to `*(A+i)`?

```
#include <stdio.h>
int sum(int *A, int n)
{
    int total = 0;
    for(int i=0 ; i < n; i++)
    {
        total += *(A+i);
    }
    A[1] = 100;
    return total;
}

int main() {
    int num[5] = {1,2,3,4,5};
    int c = sum(num, 5);
    printf("%d",num[1]);
    return 0;
}
```

Output:

```
100
```

Still the same. You can even mix `*(A+i)` and `A[1]` in the same function, as done here — they're the same operation (snippet 10).

20. Array-style parameter, pointer-arithmetic access

The fourth combination: `int A[]` in the signature, `*(A+i)` in the body.

```
#include <stdio.h>
int sum(int A[], int n)
{
    int total = 0;
    for(int i=0 ; i < n; i++)
    {
        total += *(A+i);
    }
    A[1] = 100;
    return total;
}

int main() {
    int num[5] = {1,2,3,4,5};
    int c = sum(num, 5);
    printf("%d",num[1]);
    return 0;
}
```

Output:

```
100
```

Also works. Snippets 8, 18, 19, 20 all compile to the same code: signature style (`A[]` vs `*A`) and access style (`A[i]` vs `*(A+i)`) are independent choices, and none of them causes the array to be copied.

21. Returning a local array — dangling pointer (BUG)

Now that we know a function can receive an array via a pointer, can a function *create* an array and return it via a pointer?

```
#include <stdio.h>
int *create()
{
    int num[5] = {1,2,3,4,5};
    return num;    // address of a dead local
}

int main() {
    int *p = create();
    return 0;
}
```

Output:

(nothing printed)

Compile warning: function returns address of local variable.

The program exits cleanly only because `p` is never used. `num` lives on `create`'s stack frame; when `create` returns, that frame is gone and the memory can be reused by the next function call. `p` now points to memory that belongs to nobody — a **dangling pointer**. Writing `printf("%d", p[0])` here would be undefined behaviour: it might print 1, might print garbage, might crash. We need memory whose lifetime isn't tied to a function.

22. `malloc` — memory that survives the function

How do we get memory that outlives the function that created it? Ask the system for it explicitly with `malloc`.

```
#include <stdio.h>
#include <stdlib.h>
int *create()
{
    int *num = malloc(5 * sizeof(int));
    if (num == NULL)
    {
        printf("Could not allocate");
        return NULL;
    }

    for(int i = 0; i < 5; i++)
        num[i] = (i+1);
    return num;
}

int main() {
    int *p = create();
    free(p);
    return 0;
}
```

Output:

(nothing printed)

`malloc(5 * sizeof(int))` reserves 20 bytes on the **heap** and returns their address. Heap memory isn't attached to any function's stack frame; it lives until you call `free` on it, so returning its address is safe and `p` in `main` is valid. Three habits come with `malloc`: check for `NULL` (allocation can fail), initialise the memory yourself (it starts as garbage, hence the loop), and pair every `malloc` with exactly one `free` (otherwise the memory leaks). After `free(p)`, `p` still holds the old address but must not be used.

23. Stack array vs heap array

Here are two ways to get space for 5 ints. They're used identically afterwards (`arr[i]`, `num[i]`). What's actually different?

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int arr[5];
    int *num = malloc(5 * sizeof(int));
    free(num);
    return 0;
}
```

Output:

(nothing printed)

`arr` is on the **stack**: its size must be known at compile time, and it's freed automatically when `main` returns. `num` is on the **heap**: the size is decided at run time (that `5` could be a variable read from input), it survives function returns, and it's freed only when you call `free`. Rule of thumb: stack for fixed, small, local data; heap when the size is only known at run time or the data must outlive the function. The next few snippets make "stack" and "heap" concrete.

24. Local vs global variables

So far every variable has lived inside a function. What changes if we declare one *outside* all functions? Call `bump` three times and watch the two counters.

```
#include <stdio.h>
int counter = 0;           // global: declared outside every function

void bump()
{
    int local = 0;         // local: declared inside the function
    local++;
    counter++;
    printf("local=%d counter=%d\n", local, counter);
}

int main() {
    bump();
    bump();
    bump();
    return 0;
}
```

Output:

```
local=1 counter=1
local=1 counter=2
local=1 counter=3
```

`local` is created fresh on every call — a new slot on the stack, set to 0, incremented to 1, then thrown away when `bump` returns. Three calls, three separate `local`s, so it prints `1` every time.

`counter` is a **global**: there is exactly one of it, it's created when the program starts, and it lives until the program ends, so each call sees the previous call's increment. Globals are also zero-initialised automatically (`int counter;` would still start at 0), whereas an uninitialised local holds garbage. Globals can be accessed from any function, which is convenient but also why they're easy to misuse.

25. Three regions of memory

We now have three kinds of storage: globals, locals, and `malloc`'d blocks. Are they in different places? Let's take one of each and print the addresses.

```
#include <stdio.h>
#include <stdlib.h>
int g = 0;

int main() {
    int local = 0;
    int *heap = malloc(sizeof(int));

    printf("global: %p\n", &g);
    printf("local:  %p\n", &local);
    printf("heap:   %p\n", heap);
    free(heap);
    return 0;
}
```

Output (addresses vary):

```
global: 0x10c3e8010
local:  0x7ffee4a3b9ac
heap:   0x7fb5a4c05a70
```

Three addresses in three very different parts of memory. The global sits low, in a region laid out when the program is loaded — its address never changes during the run. The local is on the **stack**, high up; the stack grows and shrinks as functions are called and return, and a local's slot is reclaimed the moment its function returns (that's the dangling-pointer bug in snippet 21). The `malloc`'d int is on the **heap**, a region that grows as you allocate and is managed by `malloc/free` rather than by function calls.

Lifetime, in one line each: global = whole program;

stack = until the function returns;

heap = until you `free` it.

26. The stack is small

If locals live on the stack, is there a limit to how much we can put there? Let's declare a large local array — 10 million ints is 40 MB.

```
#include <stdio.h>
int main() {
    int big[10000000];    // 40 MB on the stack
    big[0] = 1;
    printf("%d\n", big[0]);
    return 0;
}
```

Output (macOS):

```
zsh: segmentation fault ./a.out
```

It crashes before printing anything. The stack has a fixed, fairly small size — 8 MB by default on macOS — and asking for 40 MB of locals overflows it. This is a **stack overflow** (the same thing that happens with runaway recursion, since every call adds a frame). The compiler doesn't warn: it's a run-time limit, not a language rule. Large or variable-sized data doesn't belong on the stack.

27. Large data goes on the heap

Same 40 MB, but requested from the heap with `malloc` instead of declared as a local.

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *big = malloc(100000000 * sizeof(int));    // 40 MB on the heap
    if (big == NULL)
    {
        printf("Could not allocate");
        return 1;
    }
    big[0] = 1;
    printf("%d\n", big[0]);
    free(big);
    return 0;
}
```

Output:

```
1
```

Works. The heap is limited only by available memory (gigabytes, not megabytes), and `malloc` tells you via `NULL` if it can't satisfy the request instead of crashing. A global array (`int big[100000000];` outside `main`) would also work, since globals aren't on the stack either — but a global's size is fixed at compile time, while `malloc` lets you decide at run time.

The remaining snippets build on heap memory, which sized on demand (using `malloc`), and freed when done.

28. Growing an array by allocating a bigger one

C arrays can't be resized in place. So how do we "append" an element? Allocate a bigger block and copy.

```
#include <stdio.h>
#include <stdlib.h>
int *add(int A[], int n, int num)
{
    int *arr = malloc((n + 1) * sizeof(int));
    if (arr == NULL)
    {
        printf("Could not allocate");
        return NULL;
    }
    for (int i = 0; i < n; i++)
    {
        arr[i] = A[i];
    }
    arr[n] = num;
    return arr;
}

int main() {
    int A[5] = {10, 20, 30, 40, 50};
    int *p = add(A, 5, 60);
    int *q = add(p, 6, 70);
    free(p);
    free(q);
    return 0;
}
```

Output:

```
(nothing printed)
```

After the two calls: `A` (stack) is unchanged `{10,20,30,40,50}`; `p` (heap) is `{10,...,50,60}`; `q` (heap) is

{10,...,60,70}. Each `add` allocates `n+1` slots, copies the old `n`, writes the new value at index `n`, and returns the new block — the original is never modified. Two `malloc`s, two `free`s. `A` is not freed because it was never `malloc'd`.

29. Dynamic array from unknown-length input

Read numbers until `-1` and store them all — but we don't know how many are coming. Can we build the array up one element at a time using `add`?

```
#include <stdio.h>
#include <stdlib.h>
int *add(int A[], int n, int num)
{
    int *arr = malloc((n + 1) * sizeof(int));
    if (arr == NULL)
    {
        printf("Could not allocate");
        return NULL;
    }
    for (int i = 0; i < n; i++)
    {
        arr[i] = A[i];
    }
    arr[n] = num;
    return arr;
}

int main() {
    int *arr = NULL;
    int n = 0, num;
    while (1)
    {
        scanf("%d", &num);
        if (num == -1)
            break;
        int *newarr = add(arr, n, num);
        if (newarr == NULL)
        {
            printf("Program failed");
            break;
        }
        free(arr);
        arr = newarr;
        n++;
    }
    free(arr);
    return 0;
}
```

Input / Output:

```
3 1 4 1 5 -1
(nothing printed; arr holds {3,1,4,1,5}, n = 5)
```

Start with `arr = NULL, n = 0`. On the first iteration `add(NULL, 0, num)` copies zero elements, so it never dereferences `NULL`; and `free(NULL)` is defined to do nothing, so the first `free(arr)` is safe too. The order inside the loop matters: get `newarr` first, then `free` the old block, then reassign — freeing before copying would lose the data. Cost: every append copies everything, so `n` appends do about $n^2/2$ copies. That's the motivation for doubling-capacity growth (like `std::vector` / Python lists) later on.

30. Removing an element by walking two pointers

The reverse operation: remove the element at a given index, producing a new array of `n-1` elements. Here

we walk with pointers instead of indices. What does `p` hold at the end, and is `A` changed?

```
#include <stdio.h>
#include <stdlib.h>
int *remove_at(int *arr, int n, int index)
{
    int *newarr = malloc((n - 1) * sizeof(int));
    if (newarr == NULL)
    {
        printf("Could not allocate");
        return NULL;
    }
    int *p = newarr;
    for (int i = 0; i < n; i++)
    {
        if (i == index)
        {
            arr++;
            continue;
        }
        *p = *arr;
        arr++;
        p++;
    }
    return newarr;
}

int main() {
    int A[5] = {1, 2, 3, 4, 5};
    int *p = remove_at(A, 5, 2);
    free(p);
    return 0;
}
```

Output:

(nothing printed)

`p` holds `{1, 2, 4, 5}`; `A` is still `{1, 2, 3, 4, 5}`. Two pointers walk in step — `arr` over the source, `p` over the destination. On the index to remove, only the source advances (`arr++`) and nothing is written; everywhere else, copy and advance both. `arr++` modifies the function's own copy of the pointer, so `main`'s `A` is untouched (same reason as snippet 5). The start of the new block is kept in `newarr` because `p` has walked to the end by the time we return. One edge case is not handled: `n == 1` gives `malloc(0)`.

31. Why `int *p` rather than `int* p`?

`int* p` reads more naturally — "`int*` is the type, `p` is the variable" — and it's exactly how snippet 1 introduced it. So why does almost all C code write `int *p`? Let's declare two pointers on one line each way and see what the compiler thinks.

```
#include <stdio.h>
int main() {
    int x = 10, y = 20;
    int* p, q;      // looks like: two pointers to int
    p = &x;
    q = &y;
    printf("%d %d\n", *p, *q);
    return 0;
}
```

Output:

```
error: incompatible pointer to integer conversion assigning to 'int' from 'int *'
error: indirection requires pointer operand ('int' invalid)
```

It doesn't compile. Despite the spacing, `int* p, q;` declares `p` as `int *` and `q` as a plain `int` — the `*` binds to the variable name, not to the type. So `q = &y` tries to store an address in an `int`, and `*q` tries to dereference an `int`. To get two pointers you must write the `*` on each one:

```
int *p, *q;
```

That's the reason for the convention. `int *p` puts the `*` where it actually belongs grammatically (with `p`), so `int *p, *q` looks consistent and `int *p, q` visibly declares one pointer and one `int`. `int* p` is legal and means the same thing for a single declaration, but it suggests a grouping the compiler doesn't use, and the mistake above is easy to make. Whichever style you pick, the safe habit is one pointer declaration per line.

Summary

Snippet	Key idea
1-2	<code>&</code> gives an address, <code>*</code> (in expression) follows it; addresses are hex numbers
3-7	Pass by value copies; to modify the caller's variable, pass its address and dereference
8, 18-20	Arrays are passed as pointers; <code>A[i] ≡ *(A+i)</code> ; <code>int A[] ≡ int *A</code> in parameters
9-13, 17	Pointer indexing, arithmetic in units of elements, subtraction, <code>p++</code> , two-pointer algorithms
14-16	Print addresses with <code>%p</code> ; <code>%d/%lld</code> on a raw pointer is a type mismatch — cast first
21-27	Stack memory dies with the function and is small; globals live for the whole run; heap (<code>malloc / free</code>) is big and lives until freed
28-30	Building resizable arrays by allocate-copy-free
31	<code>int *p, *q</code> — the <code>*</code> binds to the name, so write it next to the name